

EmotionSim: A Local-First Multi-Agent Simulation System for Reproducible Swarm Experiments

Nacho Martínez (jasperan)

*Data Scientist Advocate
Oracle*

<https://github.com/jasperan>

Abstract

EmotionSim is a local-first multi-agent simulation system for studying how LLM-driven agents coordinate in high-pressure scenarios. The current implementation combines a FastAPI backend, Oracle DB 26ai Free persistence, a SvelteKit web dashboard, a Go Bubble Tea terminal UI, and an LLM router that prefers vLLM for concurrent inference while retaining Ollama fallback support. Agents maintain persona state, episodic memory, relationship memory, and an intent-driven think-plan-act-reflect loop inside a deterministic step-based engine. This paper is intentionally scoped as a systems and reproducibility paper: it documents the current architecture, deployment surface, and observability features, then reports a single benchmark run as a bounded demonstration rather than a comprehensive behavioral evaluation. On a one-step, 10-agent Airplane Crash Investigation benchmark using `gemma3:270m`, the system processed 12,549 tokens in 42.47 seconds, corresponding to 200.71 tokens per second on one machine. We close with limitations and outline a separate evaluation paper centered on repeated seeds, scenario-wide comparisons, and cohort ablations.

Keywords: Multi-Agent Systems Autonomous Agents Reproducibility Local AI Disaster Simulation

1 Introduction

Large language model agents have made it practical to simulate open-ended social interaction rather than relying only on fixed rule systems or hand-authored dialogue trees [1, 2]. That creates an opportunity for local, inspectable, and repeatable simulation environments, but only if the surrounding system makes execution, persistence, and observability manageable on developer hardware.

EmotionSim is designed around that systems problem. The current codebase ships a local-first stack that includes a FastAPI backend, Oracle-backed persistence, a SvelteKit dashboard, a Go terminal interface, built-in scenario seeding, and live event streaming for monitoring runs as they unfold. The repository currently includes nine built-in scenarios spanning floods, transportation disasters, mass-casualty response, first-contact coordination, and space-station failure.

This paper intentionally separates two questions. First, what system has been built, and how can another researcher or engineer reproduce and inspect it? Second, what scientific claims are currently supported by the experiments already run? We answer the first question in detail and keep the second narrow. Rather than presenting a broad empirical claim about social realism or agent validity, we report one benchmark artifact as a bounded demonstration of the system in operation.

2 Current Implementation

2.1 Service Architecture

EmotionSim is implemented as a multi-surface application with shared backend orchestration:

```

SvelteKit dashboard (:3000) / Go Bubble Tea TUI
  -> REST + WebSocket
FastAPI backend (:8000)
  -> SimulationEngine
  -> MessageBus / ConversationManager / datalake logger
  -> LLMRouter (vLLM primary, Ollama fallback)
  -> Oracle DB 26ai Free

```

The FastAPI application initializes the database, auto-seeds built-in scenarios when the scenario table is empty, and attempts to resume active runs on startup. The web client and terminal UI consume the same backend APIs and event stream, which keeps the simulation logic centralized while exposing multiple operator views.

2.2 Agent Model

The implementation uses stateful agents rather than stateless prompt templates. Human agents carry both stable persona fields and mutable simulation state:

```

1 class Persona(BaseModel):
2     name: str
3     age: int
4     sex: Literal["male", "female", "non-binary"]
5     occupation: str
6
7     # Big Five traits on a 1-10 scale
8     openness: int
9     conscientiousness: int
10    extraversion: int
11    agreeableness: int
12    neuroticism: int
13
14    # Additional behavioral modifiers
15    risk_tolerance: int
16    empathy_level: int
17    leadership: int
18
19    # Dynamic state
20    stress_level: int
21    health: int
22    inventory: list[str]
23    location: str

```

Listing 1: Simplified Persona Schema

Beyond the schema above, each agent also maintains episodic memory, relationship memory, intent memory, and action feedback buffers. Those components support the current cognitive pipeline, where an agent can assess the world, form or reuse a plan, act, and then reflect on whether the plan should continue.

2.3 Execution Model

At runtime, the simulation proceeds in deterministic steps. Environment agents update the world first, human agents then act either in scene-based groupings or sequential order depending on configuration, and the engine persists state plus WebSocket events after each step. In the current codebase, scene mode, governance gates, fallback routing, and token-budget-based conclusion prompts are configurable features of the execution loop.

```

1 async def execute_step():
2     tick_environment_agents()
3     run_human_scenes_or_single_agent_ticks()
4     run_reaction_round()
5     persist_state_and_emit_events()

```

Listing 2: High-Level Step Structure

This design is meant to preserve inspectability. A run can be observed through the web dashboard, the Go TUI, and persisted records in the datalake tables, rather than only through transient console logs.

3 Reproducibility Surface

The current repository exposes several reproducibility hooks that matter for systems work.

First, deployment is local-first. The repository includes a Docker Compose stack that brings up Oracle DB 26ai Free, the backend, and the frontend together. It also supports direct local development through the backend package, the SvelteKit frontend, and the Go TUI.

Second, inference configuration is explicit. The default settings prefer vLLM with Qwen/Qwen3.5-4B for concurrent agent execution, while retaining Ollama settings and fallback models for local runtimes that do not use vLLM. That means a paper or benchmark report should always state the active backend, model, and routing configuration rather than treating “the model” as an implicit constant.

Third, the repository already contains a benchmark harness and a checked-in benchmark artifact. The script `backend/tools/benchmark_gemma.py` executes a one-step, 10-agent run and records latency plus token metrics. The checked-in benchmark result used in this paper comes from that harness. Even though the default runtime path now prefers vLLM and Qwen, the benchmark artifact remains useful as a concrete, reproducible demonstration of the system under a specific Ollama-based configuration.

4 Demonstration Benchmark

4.1 Scope of the Demonstration

This section is deliberately narrow. Its purpose is to show that the current implementation can execute a local multi-agent step, persist state, and expose measurable throughput on one machine. It is not intended to validate social realism, prove general scalability, or establish that one scenario’s outcome generalizes across the scenario library.

4.2 Setup

The benchmark artifact in the repository uses the following setup:

- Scenario: Airplane Crash Investigation
- Agent count: 10
- Duration: 1 simulation step
- Model: `gemma3:270m`
- Measurement harness: `backend/tools/benchmark_gemma.py`

Table 1: Single-Run Demonstration Benchmark (10 Agents, 1 Step)

Metric	Result
Total Duration (Wall Time)	42.47 sec
Total LLM Requests	9
Total Tokens Generated	8,525
Total Tokens Processed	12,549
Average Latency	4.72 sec
System Throughput	200.71 tokens/sec

The measured result supports a constrained systems claim: EmotionSim can run a small, locally hosted, observable multi-agent step on commodity hardware while maintaining a non-trivial token throughput. That is useful evidence for the viability of the software stack. It is not, by itself, evidence that the platform has solved large-scale social simulation, nor that the same throughput would hold across longer runs, larger agent populations, different models, or different hardware.

5 Limitations and Scope

Several limitations are important if this paper is read as evidence rather than as a systems description.

First, the agents are simulated agents, not validated human proxies. The system is useful for exploratory experiments and tooling, but this paper does not establish that the behaviors produced by the agents correspond to real human social dynamics.

Second, outcomes are sensitive to prompt construction, model choice, routing policy, and scenario authoring. The current codebase includes multiple configurable inference paths, including vLLM as the preferred concurrent backend and Ollama as a fallback path. Results should therefore be interpreted as configuration-dependent.

Third, the benchmark in Table 1 is a single-run artifact measured on one machine. It demonstrates that the implementation works in that configuration; it does not prove general scalability, stable performance under long horizons, or cross-hardware reproducibility.

Fourth, this paper does not present repeated-seed experiments, cross-scenario comparisons, or ablations on personality diversity, governance, scene mode, or routing strategy. Those are exactly the kinds of questions that require a separate evaluation-oriented study.

6 Toward a Separate Evaluation Paper

To keep this manuscript honest, the broader experimental claims should be moved into a follow-on paper rather than implied here.

That second paper should evaluate the scenario library across repeated seeds and explicit cohort ablations. A reasonable design would include:

1. multi-scenario evaluation across the nine built-in scenarios, rather than a single case;
2. repeated-seed runs per scenario to characterize variance rather than anecdote;
3. cohort ablations comparing diverse-personality groups against more homogeneous agent populations;
4. system ablations over inference backend, fallback routing, scene mode, and governance settings; and
5. standardized metrics such as task completion, survival, consensus latency, message volume, trust changes, and intervention frequency.

In other words, the current paper should remain a demo and systems paper, while the future paper becomes the place to answer behavioral questions about generalization, robustness, and the effects of personality diversity.

7 Conclusion

EmotionSim demonstrates that a local-first, inspectable multi-agent simulation system can be packaged into a coherent developer workflow with backend persistence, live dashboards, configurable inference routing, and reproducible benchmark tooling. The current repository is strong enough to support systems discussion and bounded demonstrations. What it does not yet justify is a broad claim that local agent swarms faithfully model social behavior in general. Treating this manuscript as a systems paper, and moving broader behavioral validation into a dedicated follow-on evaluation paper, is the more defensible framing.

Code Availability

The code, built-in scenarios, and benchmark tooling are available in the public repository below.

<https://github.com/jasperan/emotion-engine>

References

- [1] Joon Sung Park, Joseph C. O’Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. Generative agents: Interactive simulacra of human behavior. *arXiv preprint arXiv:2304.03442*, 2023.
- [2] Zhiheng Xi, Wenxiang Chen, Xin Guo, Wei He, Yiwon Ding, Boyang Hong, Ming Zhang, et al. The rise and potential of large language model based agents: A survey. *arXiv preprint arXiv:2309.07864*, 2023.